FIAP ON

DJANGO WEB FRAMEWORK

DJANGO 101

ANDREY MASIERO

1

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1.1 – Criação do ambiente virtual com o Python 6
Figura 1.2 – Ativação do ambiente virtual 6
Figura 1.3 – Django instalado com sucesso 7
Figura 1.4 – Opções de comandos para o django-admin 7
Figura 1.5 – Opções do comando startproject do django-admin 8
Figura 1.6 – Estrutura do projeto Django criado 9
Figura 1.7 – Executando o comando para rodar o servidor do Django 10
Figura 1.8 – Consultando o projeto no navegador 10
Figura 1.9 – Tela de login da parte administrativa do Django 11
Figura 1.10 – Migração realizada com sucesso 12
Figura 1.11 – Criação do superuser 12
Figura 1.12 – Tela inicial da área administrativa do Django 13
Figura 1.13 – Tela inicial da área administrativa em português 13

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1.1 – Configuração do sistema para português 13
Código-fonte 1.2 – Adição dos aplicativos criados no sistema.....14

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 DJANGO 5
1.1 Introdução 5
1.2 Criando um ambiente Virtual 5
1.3 Instalando o Django 6
1.4 Criação e execução do projeto 7
1.5 Acessando a área administrativa 11
1.6 Criando e vinculando Apps ao Sistema 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 DJANGO

1.1 Introdução

O Django é um framework de desenvolvimento web escrito em Python que permite um desenvolvimento rápido, limpo e pragmático. O objetivo dele é que o desenvolvedor foque seu desenvolvimento nas regras de negócio e não precise reinventar a roda. Com ele é possível conceber desenvolvimentos rápidos de aplicações do conceito à completude total. (Mais rápido que o Superman. Ok essa parte é por minha conta.)

Outro ponto interessante do Django é a segurança. O framework leva o desenvolvimento seguro muito a sério e tenta conduzir o desenvolvedor a evitá-los a qualquer custo. Além disso, as aplicações desenvolvidas com Django são totalmente escaláveis. Então é bem fácil de entender como criar ou adicionar aplicações em seu ecossistema.

Vamos agora preparar o ambiente para instalarmos o Django e iniciarmos o projeto. É importante que você já tenha instalado o Python na sua última versão. Ele é encontrado no site https://www.python.org/. Para Windows, recomendo que instale a versão mais atual disponível através do Anaconda. Você encontra a distribuição Anaconda no link https://www.anaconda.com/products/individual.

1.2 Criando um ambiente Virtual

Em vez de instalar todas as ferramentas que você deseja diretamente em sua máquina e sobrecarregar a memória com informações que você não utiliza frequentemente, com o Python é possível criar Ambientes Virtuais. Dessa maneira, você deixa instalados apenas na sua aplicação as bibliotecas e os módulos que são fundamentais para a execução da sua aplicação. E isso fica concentrado em um único local. Para criar o ambiente virtual, acesse o terminal do Windows e digite o seguinte comando python3 -m venv <caminho da aplicacao>\venv. Após essa etapa, pressione enter e terá o resultado similar ao apresentado na figura “Criação do ambiente virtual com o Python”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
D:\django\library-app (main -> origin)
  python -m venv D:\django\library-app\venv

D:\django\library-app (main -> origin)
  ls
LICENSE  README.md  venv/

D:\django\library-app (main -> origin)
  ls venv\
Include/  Lib/  pyvenv.cfg  Scripts/

D:\django\library-app (main -> origin)
```

Figura 1.1 – Criação do ambiente virtual com o Python
Fonte: Elaborado pelo autor

Após a criação do ambiente virtual é preciso ativá-lo para que as bibliotecas e os comandos sejam todos dados nesse repositório. Execute o seguinte comando na pasta do projeto, pelo terminal: venv\Scripts\activate.bat. O terminal deve aparecer agora com a palavra venv entre parênteses. Isso indica que os comandos dados serão executados dentro do ambiente criado. Confira com a figura “Ativação do ambiente virtual”.

```
D:\django\library-app (main -> origin)
  venv\Scripts\activate.bat

D:\django\library-app (main -> origin)
(venv)
```

Figura 1.2 – Ativação do ambiente virtual
Fonte: Elaborado pelo autor

Nesse momento a aplicação está pronta para começar a instalação do Django no projeto.

1.3 Instalando o Django

Para instalar o Django agora precisamos apenas executar o seguinte comando no terminal: pip install django e aguardar a instalação ser realizada. A mensagem de sucesso da instalação do Django deve ser similar a apresentada na figura “Django instalado com sucesso”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Installing collected packages: pytz, asgiref, sqlparse, django
Successfully installed asgiref-3.2.10 django-3.1.2 pytz-2020.1 sqlparse-0.3.1
You are using pip version 19.0.3, however version 20.2.3 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

D:\django\library-app (main -> origin)
(venv)
```

Figura 1.3 – Django instalado com sucesso
Fonte: Elaborado pelo autor

Com o Django instalado, podemos iniciar o processo de criação do projeto e seus respectivos modelos.

1.4 Criação e execução do projeto

A partir do momento que temos o Django instalado, podemos utilizar a ferramenta django-admin para criarmos projetos entre outras coisas em nosso sistema. Vamos digitar no terminal a opção django-admin help para identificar quais as ações que podemos executar com ele. A figura “Opções de comandos para o django-admin” apresenta o resultado dessa consulta.

```
D:\django\library-app (main -> origin)
(venv)   django-admin help

Type 'django-admin help <subcommand>' for help on a specific subcommand.

Available subcommands:

[django]
    check
    compilemessages
    createcachetable
    dbshell
    diffsettings
    dumpdata
    flush
    inspectdb
    loaddata
    makemessages
    makemigrations
    migrate
    runserver
    sendtestemail
    shell
    showmigrations
    sqlflush
    sqlmigrate
    sqlsequencereset
    squashmigrations
    startapp
    startproject
    test
    testserver
```

Figura 1.4 – Opções de comandos para o django-admin
Fonte: Elaborado pelo autor

Veja o destaque na figura “Opções de comandos para o django-admin” para o comando startproject. Esse é o comando que devemos executar para criar o projeto. Antes de executá-lo, vamos ver as opções de projetos que podemos ter com ele. A

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

figura “Opções do comando startproject do django-admin” apresenta as opções do startproject, após a execução do: django-admin help startproject.

```
D:\django\library-app (main -> origin)
(venv)   django-admin help startproject
usage: django-admin startproject [-h] [--template TEMPLATE]
                                 [--extension EXTENSIONS] [--name FILES]
                                 [--version] [-v {0,1,2,3}]
                                 [--settings SETTINGS]
                                 [--pythonpath PYTHONPATH] [--traceback]
                                 [--no-color] [--force-color]
                                 name [directory]

Creates a Django project directory structure for the given project name in the
current directory or optionally in the given directory.

positional arguments:
  name                  Name of the application or project.
  directory             Optional destination directory

optional arguments:
  -h, --help            show this help message and exit
  --template TEMPLATE   The path or URL to load the template from.
  --extension EXTENSIONS, -e EXTENSIONS
                        The file extension(s) to render (default: "py").
                        Separate multiple extensions with commas, or use -e
                        multiple times.
  --name FILES, -n FILES
                        The file name(s) to render. Separate multiple file
                        names with commas, or use -n multiple times.
  --version             show program's version number and exit
  -v {0,1,2,3}, --verbosity {0,1,2,3}
                        Verbosity level; 0=minimal output, 1=normal output,
                        2=verbose output, 3=very verbose output
  --settings SETTINGS   The Python path to a settings module, e.g.
                        "myproject.settings.main". If this isn't provided, the
                        DJANGO_SETTINGS_MODULE environment variable will be
                        used.
  --pythonpath PYTHONPATH
                        A directory to add to the Python path, e.g.
                        "/home/djangoprojects/myproject".
  --traceback           Raise on CommandError exceptions
  --no-color            Don't colorize the command output.
  --force-color         Force colorization of the command output.
```

Figura 1.5 – Opções do comando startproject do django-admin
Fonte: Elaborado pelo autor

Podemos, além de criar o projeto, determinar a versão, uso de *templates*, extensões, configurações de módulos entre outras coisas. Mas, agora, chegou a hora de criarmos o projeto de nossa biblioteca, em que vamos trabalhar ao longo do nosso curso. No terminal execute o comando: django-admin startproject library. Após a execução do comando, podemos conferir a estrutura de arquivos criado através do Windows Explorer ou qualquer outro gerenciador de arquivos. A figura “Estrutura do

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

projeto Django criado” apresenta a estrutura inicial de arquivos de um projeto em Django.

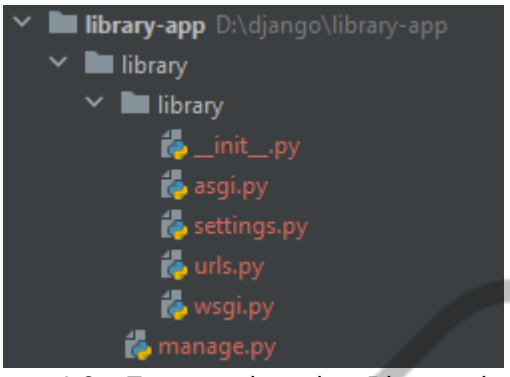

Figura 1.6 – Estrutura do projeto Django criado
Fonte: Elaborado pelo autor

Os scripts criados possuem as seguintes finalidades:

- **manage.py**: responsável pelos processos de automação de projeto, será útil para iniciar o servidor local, realizar novos mapeamentos de entidades, entre outros.
- **__init__.py**: serve para informar que este projeto pode ser embutido como um pacote.
- **settings.py**: contém as configurações globais de projeto, nele são especificados os módulos específicos do Django que serão utilizados, conectores do banco de dados e demais configurações de alto nível do sistema.
- **urls.py**: contém as rotas do projeto, especificando inclusive as regras de navegação.
- **wsgi.py**: é o ponto de acesso para redirecionamento a partir de um servidor WSGI.

Todas as configurações no Django são realizadas como se fossem código Python. Os parâmetros e estruturas guardam as informações como strings. A estrutura inicial de projeto no Django é extensa e centralizada, o que facilita as configurações dos projetos.

Vamos executar o projeto e verificar se está tudo okay antes de continuarmos. Para isso, basta executar o comando: python manage.py runserver para executar o projeto localmente em um webserver criado pelo próprio Django. A figura

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

“Executando o comando para rodar o servidor do Django” apresenta o resultado após rodar o comando no terminal.

```
D:\django\library-app (main -> origin)
(venv)   cd library\

D:\django\library-app\library (main -> origin)
(venv)   python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you
apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
October 05, 2020 - 20:41:52
Django version 3.1.2, using settings 'library.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Figura 1.7 – Executando o comando para rodar o servidor do Django
Fonte: Elaborado pelo autor

Perceba na figura “Executando o comando para rodar o servidor do Django” a mensagem “Starting development server at http://127.0.0.1:8000”, isso significa que basta acessarmos o navegador com esse endereço para ver o resultado obtido com o projeto em execução. O endereço 127.0.0.1 pode também ser substituído pelo hostname localhost. Acesse o endereço em seu navegador, o resultado deve ser parecido com o apresentado na figura “Consultando o projeto no navegador”.

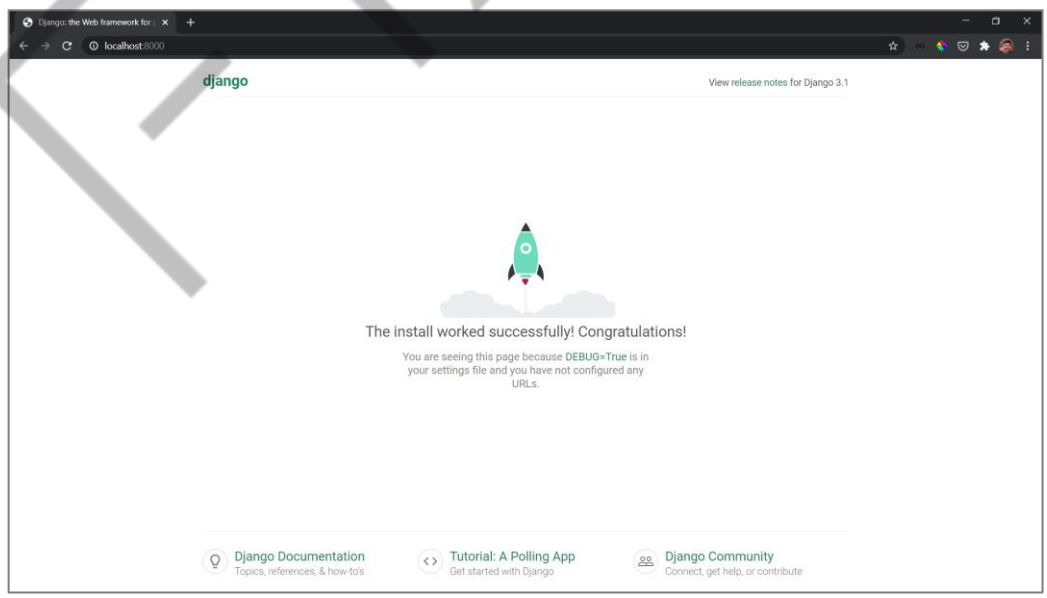

Figura 1.8 – Consultando o projeto no navegador
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além desse acesso, que representa o site que o usuário deve ver, o Django disponibiliza um site de administração do sistema. Para isso, basta colar a informação “/admin” após o endereço do projeto no navegador. Vamos ver isso na próxima seção.

1.5 Acessando a área administrativa

Acessando o endereço: http://localhost:8000/admin temos acesso à área administrativa. Para acessar as funcionalidades, é necessário que seja efetuado o login de acesso, conforme apresentado na figura “Tela de login da parte administrativa do Django”.

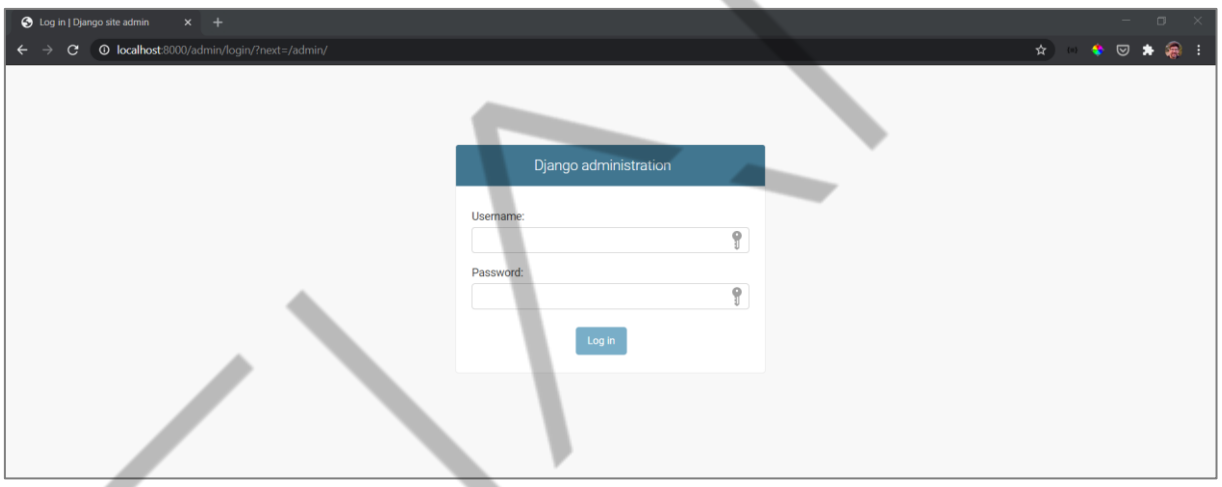

Figura 1.9 – Tela de login da parte administrativa do Django
Fonte: Elaborado pelo autor

Contudo, não foi realizada em nenhum momento a criação de um usuário para o projeto. Através do script manage.py é possível criar um superuser. Mas, antes que isso ocorra, é necessário realizar um processo de migração dos modelos predefinidos para o banco de dados. Nesse momento, você pode estar se perguntando: qual banco de dados? Não instalamos nenhum banco de dados! E você tem total razão. Só que o Django já vem por padrão com um banco de dados mais simples, porém comporta tudo o que precisamos para realizar esse projeto. O banco de dados padrão do Django é o SQLite.

Para realizarmos a migração precisamos executar o script através do comando python manage.py migrate. Veja o resultado na figura “Migração realizada com sucesso”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Figura 1.10 – Migração realizada com sucesso
Fonte: Elaborado pelo autor

Agora o sistema está apto para criarmos o superusuário, que será o administrador do nosso sistema. Execute o comando: python manage.py createsuperuser. Ele solicitará o nome do usuário, e-mail e senha. A senha deve ter pelo menos 8 caracteres e misturar letras maiúsculas e minúsculas, números e caracteres especiais. É possível criar com uma senha totalmente numérica, como eu defini no exemplo, de 1 a 6. Mas mensagens de aviso vão aparecer na tela, conforme a figura “Criação do superuser”.

```
D:\django\library-app\library (main -> origin)
(venv)   python manage.py createsuperuser
Username (leave blank to use 'andre'): andrey
Email address: profandrey.masiero@fiap.com.br
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Figura 1.11 – Criação do superuser
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nesse momento, podemos voltar ao navegador que está na tela de autorização de acesso e inserir o usuário e a senha que criamos no passo anterior. Caso o login seja realizado com sucesso, teremos o resultado da figura “Tela inicial da área administrativa do Django”.

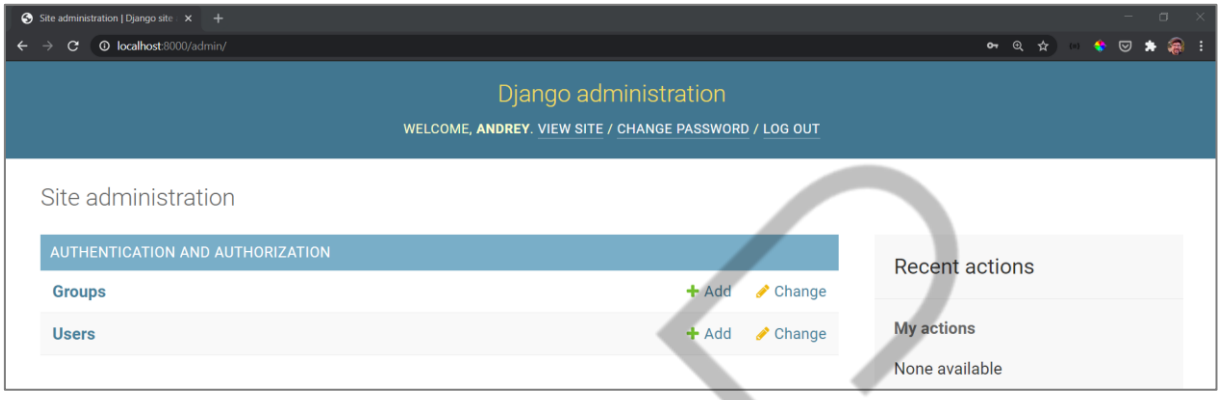

Figura 1.12 – Tela inicial da área administrativa do Django
Fonte: Elaborado pelo autor

Nessa tela podemos criar usuários e grupos de acesso em nosso sistema. Entretanto, as informações estão em inglês. Vamos configurar para a língua portuguesa. Acesse o arquivo: settings.py. Altere as informações sobre LANGUAGE_CODE e TIME_ZONE para o código abaixo.

```
LANGUAGE_CODE = 'pt-br'
TIME_ZONE = 'America/Sao_Paulo'
```

Código-fonte 1.1 – Configuração do sistema para português
Fonte: Elaborador pelo autor

Agora o sistema ficou com as informações em português, conforme a figura “Tela inicial da área administrativa em português”.

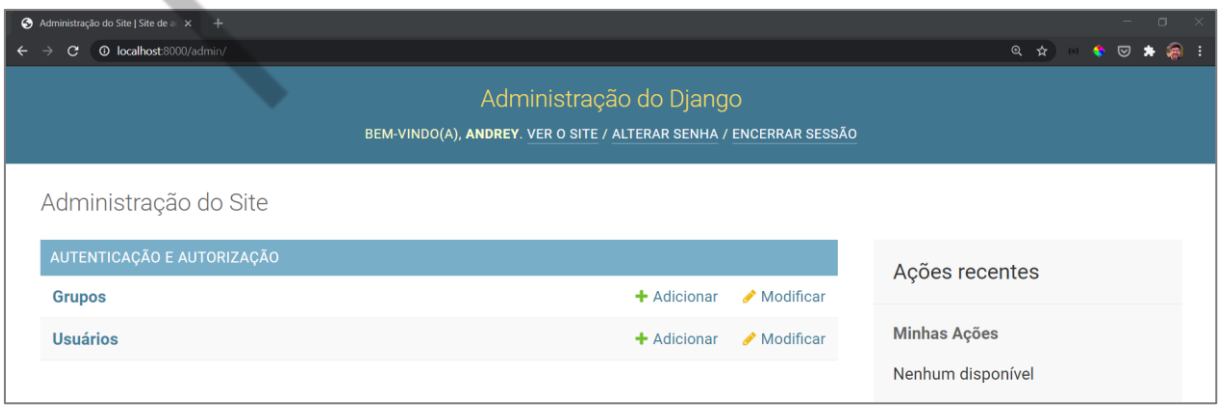

Figura 1.13 – Tela inicial da área administrativa em português
Fonte: Elaborado pelo autor

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Existem muitas possibilidades para a área administrativa do site, mas não vamos entrar em detalhes nesse momento do curso. Mais adiante, abordaremos as funcionalidades da área de administração, inclusive sobre a criação de novos usuários e grupos. Antes de finalizarmos o capítulo, falta criar os aplicativos que serão responsáveis pelos livros e leitores da nossa biblioteca.

1.6 Criando e vinculando Apps ao Sistema

Para manter organizada a arquitetura do Django, cada objeto de nosso sistema é tratado como um aplicativo. Em nosso caso, temos dois objetos importantes para nosso sistema, os livros e os leitores que pegaram emprestados os livros. Vamos começar pelos livros. Execute o comando: python manage.py startapp livros. Na sequência execute o mesmo comando para os autores.

Agora é necessário registrar ambos os aplicativos no sistema. Dentro da pasta library, acesse o arquivo settings.py. Procure pela lista INSTALLED_APPS e adicione o nome os aplicativos como no código demonstrado a seguir.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'livros',
    'autores'
]
```

Código-fonte 1.2 – Adição dos aplicativos criados no sistema
Fonte: Elaborado pelo autor

Pronto, nesse momento concluímos a criação e configuração do projeto. Os próximos passos são as criações de modelos e dos demais objetos das camadas do padrão arquitetural MVC. Mas isso é história para outro momento.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

Django. **Documentação do Django Web Framework**. [s.d.]. Disponível em: https://www.djangoproject.com/. Acesso em: 05 out. 2020.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com